Experiment - 13

Case Study on Limitations of Low-Code Platforms for a Complex Enterprise Use Case

6th October 2026

SIVARATHINAM S
241401102 B.Tech CSBS

AIM

To study the limitations of low-code platforms when they are applied to a complex enterprise-level Hotel Booking system, and to understand when pro-code extensions or a hybrid approach become necessary.

INTRODUCTION

Low-code platforms let applications be built mainly through visual tools rather than large amounts of hand-written code. Forms, reports and dashboards are assembled from ready-made components, behaviour is configured through workflows and rules, and a little scripting is available where needed. This shortens development time and allows business users to take part in building applications.

Such platforms suit standard business applications well. As an enterprise grows in users, data volume, process complexity, integrations, security demands and customisation needs, the built-in features of a platform may stop being enough on their own. In Experiment 14, a Hotel Booking application was extended with a custom Deluge function. This case study builds on it by examining the limitations that appear when the same idea is scaled to an enterprise hotel group.

Case Study: Enterprise Hotel Booking Management System

A hotel group operating several properties needs one system to manage its room reservations and related operations. The scenario is a general illustration and does not use measured figures. The system must serve the following users and handle the following functions:

Category	Items
Users	Guests, front-desk staff, housekeeping staff, revenue managers and administrators
Data and operations	Room records, availability, reservations, check-in and check-out, booking history
Supporting functions	Notifications, payments, room inventory and rate plans
Enterprise integration and insight	CRM, property management system, channel manager / online travel agents, analytics and reporting

Table 1: Users and Functions of the Hotel Booking System

Figure 1 shows the architecture of this system on a low-code platform. It is arranged as six groups connected from left to right:

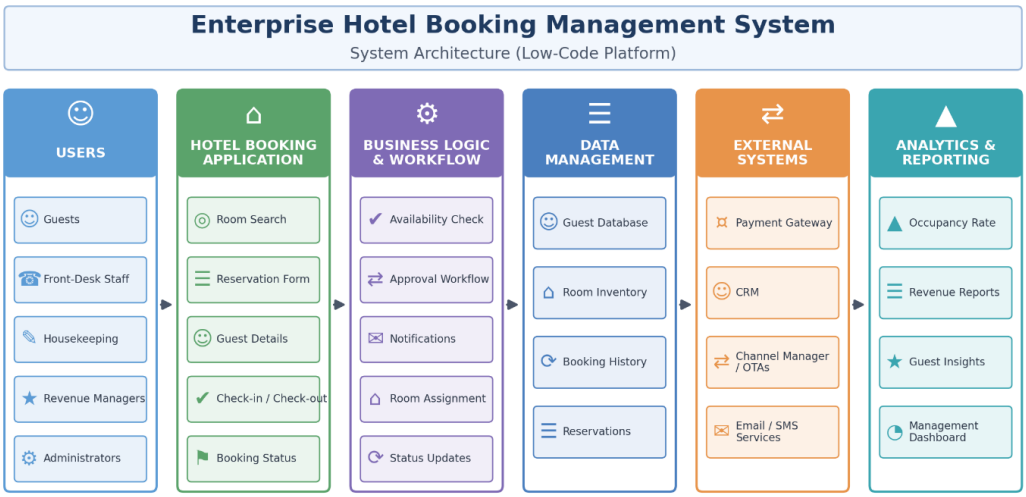


Figure 1: Enterprise Hotel Booking Management System Architecture

- **Users:** guests, front-desk staff, housekeeping staff, revenue managers and administrators who work with the system.
- **Hotel Booking Application:** the user-facing part, covering room search, reservation entry, guest details, check-in/check-out and booking status.
- **Business Logic & Workflow:** availability checks, approvals, notifications, room assignment and status updates that carry a booking through its life cycle.
- **Data Management:** storage of guest records, room inventory, reservations and booking history.
- **External Enterprise Systems:** the payment gateway, CRM, channel manager / online travel agents and property management system with which the application exchanges data.
- **Analytics & Reporting:** occupancy rate, revenue reports, guest insights and a management dashboard for decision makers.

The groups work in sequence. Users enter and view data through the application, the workflow layer processes it, the data layer stores and supplies it, external systems send or receive related information, and the reporting layer summarises the results.

How a Low-Code Platform Can Support the System

A large part of the system is standard application functionality, which is what low-code platforms are designed for:

Capability	Use in the Hotel Booking System
Forms	Capturing reservations, guest details and room information
Reports	Viewing bookings, occupancy, booking history and guest reports
Workflows	Moving a booking through confirmation, approval, check-in and check-out
Business process automation	Automatic actions, for example marking a room occupied when a guest checks in
Data management	Storing guest, room and reservation records with create, read, update and delete operations
User interfaces	Web and mobile screens for guests, front-desk staff and managers
Notifications	Sending booking confirmations and reminders by email or message
Basic integrations	Connecting to common services through built-in connectors

Table 2: Low-Code Capabilities Used in the System

Limitations at Enterprise Scale

As the system grows into a complex enterprise application, the following limitations may become significant.

1. Performance and Scalability

With many properties, rooms and guests, searches and reports over large data sets can become slow, especially in peak season. The platform manages the infrastructure and sets its own limits, so the options for tuning performance and scaling are restricted.

2. Complex Business Logic

Rules such as dynamic room pricing, overbooking control and optimised room allocation are hard to express through visual workflows and standard scripting. They usually need custom algorithms and more advanced programming.

3. System Integration

Connecting to property management, ERP, CRM and payment systems, online travel agents and older legacy systems can be difficult. Built-in connectors may not cover every system or data format, and real-time synchronisation of room availability may require custom interfaces.

4. Customization

Standard components provide common screens and layouts. A branded guest booking experience, or a feature the platform does not offer, may go beyond what these components can do.

5. Security and Governance

A hotel group holds personal and payment-related data and must meet compliance rules. The access controls of a platform may need to be supplemented, and many applications built by different teams must be governed carefully.

6. AI and Advanced Technology

Demand forecasting, personalised offers and similar intelligent features usually depend on external services or custom code, because a low-code platform may not provide them directly.

7. Vendor / Platform Dependency

The application relies on the features, restrictions, pricing and plan limits of one platform. Moving to another platform later can be difficult.

8. Data Management

Large booking histories, complex relationships between records and heavy batch processing or data transformation may exceed what the platform's data model and tools can handle comfortably.

Figure 2 summarises how these limitations arise. A basic low-code application is quick to build and suits modest needs. Growing enterprise requirements, such as more guests and data, complex pricing and processes, and many integrations, lead to enterprise-scale challenges in performance, security, governance and customisation. The eight limitations above are the result.

2140102

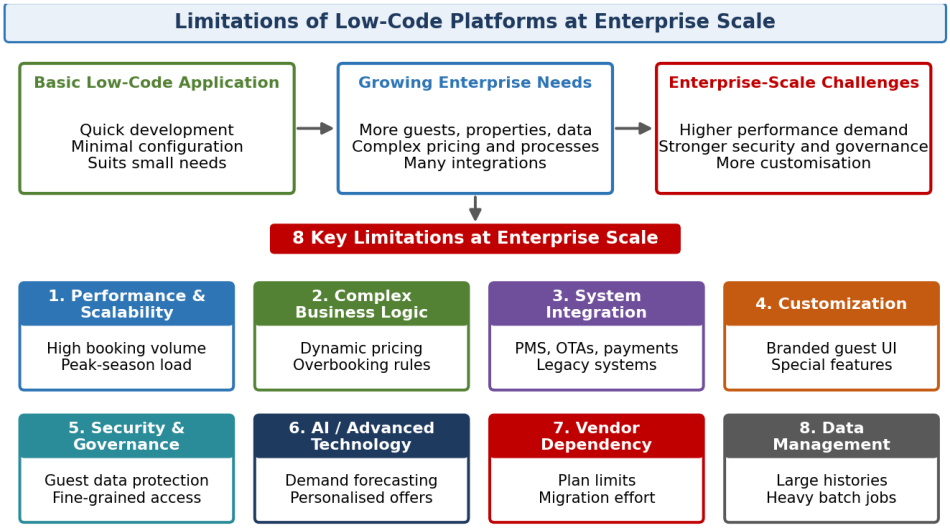


Figure 2: Limitations of Low-Code Platforms at Enterprise Scale

The relevance of each limitation to the Hotel Booking system is given below.

Limitation	Relevance to the Case Study
Performance & Scalability	Growing numbers of guests, rooms and bookings, and seasonal peaks in searches
Complex Business Logic	Dynamic pricing, overbooking and room-allocation rules that grow more complex
System Integration	Payment gateway, CRM, channel manager and property management system
Customization	A branded guest booking journey and tailored screens for staff
Security & Governance	Protection of guest and payment data; different access for each user role
AI / Advanced Technology	Demand forecasting and personalised offers built on booking history
Vendor Dependency	Reliance on one platform for application, workflow and reporting features
Data Management	Large booking histories with relationships to guests, rooms and payments

Table 3: Relevance of Each Limitation to the Case Study

Low-Code vs Pro-Code Requirements

Neither approach is universally better. The suitability of each depends on the requirements of the application, and the comparison below shows the typical role of each.

Area	Low-Code Approach	When Pro-Code May Be Required
User interface	Standard screens and forms from predefined components	Highly branded or specialised user experiences
Business logic	Visual workflows, rules and simple scripting	Complex rules and advanced processing
Data management	Built-in data model with standard operations	Very large data sets and complex data processing
Integrations	Built-in connectors and standard interfaces	Legacy, real-time or specialised integrations
Automation	Rules, schedules and workflow-based automation	High-volume or specialised automation
AI / ML	Basic features offered by the platform	Custom models and machine-learning pipelines
Security	Role- and permission-based access control	Specific compliance and fine-grained control
Scalability	Suitable where the platform handles expected load	High volumes and demanding performance

Table 4: Low-Code and Pro-Code Compared

Hybrid Approach

Organisations can combine the two approaches. Low-code is used to build standard application functionality quickly, while custom code, APIs and integrations handle requirements that exceed the built-in capabilities of the platform. The structure has three layers:

Low-Code Application → Pro-Code Extensions → Enterprise Systems

Figure 3 shows this architecture. Users access the system at the top, and information flows in both directions between the layers.

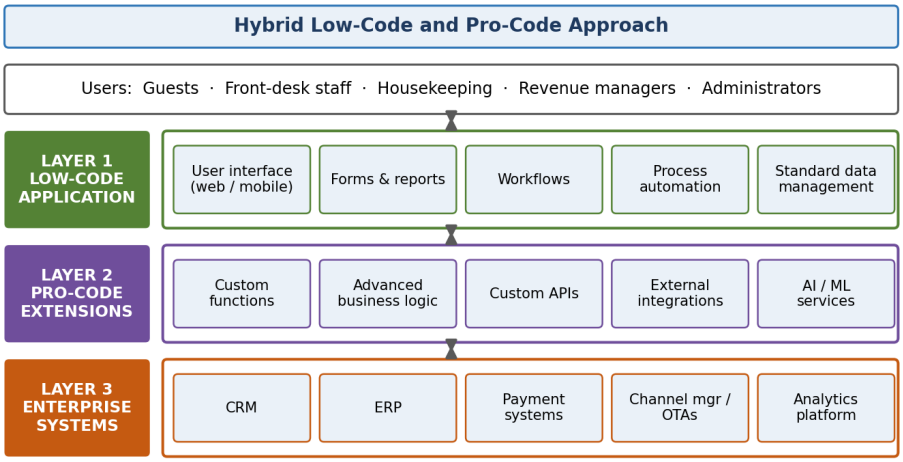


Figure 3: Hybrid Low-Code and Pro-Code Enterprise Architecture

Layer	Role
Layer 1 – Low-Code Application	Rapid development of standard needs: user interface, forms and reports, workflows, process automation and standard data management
Layer 2 – Pro-Code Extensions	Custom logic and specialised capabilities: custom functions, advanced business logic, custom APIs, external integrations and AI / ML services
Layer 3 – Enterprise Systems	Core systems that supply data and services: CRM, ERP, payment systems, channel manager, external databases and analytics platforms

Table 5: Layers of the Hybrid Architecture

The reasons for a hybrid approach are faster development, flexible customisation, integration with existing systems, support for complex business logic and the ability to go beyond platform limits. In Zoho Creator, a custom Deluge function (Experiment 14) or an external service called through an API are examples of pro-code extensions to a low-code application.

When Pro-Code Extension May Become Necessary

- **Complex business logic:** when pricing, overbooking or allocation rules cannot be shown with standard workflows.
- **Specialised integrations:** when the system must connect to legacy or other systems that have no suitable connector.
- **Custom APIs:** when services must be exposed to or consumed from other systems in a way the platform does not support.
- **Advanced AI/ML:** when forecasting or personalisation needs custom models.
- **High-performance processing:** when large batches or real-time processing exceed what the platform handles efficiently.
- **Specialised user interfaces:** when the guest experience needs more than standard components.
- **Requirements beyond platform capabilities:** when a needed feature is simply not offered by the chosen platform.

Case Study Analysis

For the Hotel Booking system, a low-code platform can be used effectively for the standard parts: reservation forms, guest and room details, booking reports, approval and check-in workflows, notifications and basic dashboards. These match what low-code platforms are designed to provide.

As the group grows, demands increase. Larger numbers of guests and bookings raise performance and data concerns; payment, CRM and channel-manager links raise integration concerns; different user roles and guest data raise security and governance concerns; and features such as demand forecasting may need external services. In such cases, extensions, integrations or custom development are needed alongside the low-code application. Low-code is therefore valuable for building standard business applications quickly, while increasingly complex requirements call for a hybrid approach chosen according to the needs of the application.

CONCLUSION

The case study showed that low-code platforms face limitations in complex enterprise applications, namely performance and scalability, complex business logic, system integration, customisation, security and governance, AI and advanced technology, vendor dependency and data management. For the Enterprise Hotel Booking Management System, standard functions can be built quickly with low-code, while demanding requirements may need pro-code extensions. A hybrid low-code and pro-code approach therefore meets enterprise needs best, and a platform's limitations should be understood before choosing a development approach.